

UNIVERSITATEA DIN BUCUREȘTI

FACULTATEA DE MATEMATICĂ ȘI INFORMATICĂ



SPECIALIZAREA INFORMATICĂ

Lucrare de licență

COURSEHUB – PLATFORMĂ WEB PENTRU CONECTAREA PROFESORILOR CU ELEVII

Absolvent

Avramescu Isaac Sebastian

Coordonator științific

Conf. Univ. Dr. Marius Iulian Mihăilescu

București, septembrie 2026

Rezumat

Lucrarea de față prezintă proiectarea și dezvoltarea aplicației web CourseHub, o platformă care conectează profesori și elevi într-un spațiu comun de învățare. Obiectivul principal este construirea unui sistem în care un utilizator își poate crea un profil ca profesor sau ca elev, poate căuta un partener potrivit după materie, nivel și rol, poate iniția o colaborare printr-o cerere de colaborare și poate comunica ulterior în timp real, inclusiv prin partajarea de fișiere și materiale didactice. Platforma a fost dezvoltată folosind ASP.NET Core 8, Entity Framework Core și SQL Server, cu comunicare în timp real realizată prin SignalR. Printre funcționalitățile implementate se numără autentificarea securizată prin cod OTP trimis pe email, un sistem de roluri bazat pe Identity Roles cu panou de administrare dedicat, protecția formularului de înregistrare prin reCAPTCHA, încărcarea de avatare și fișiere validate atât din punct de vedere al tipului, cât și al dimensiunii, precum și o interfață disponibilă integral în limbile română și engleză. Rezultatele obținute arată o platformă funcțională, cu accent pe securitate, experiența utilizatorului și posibilitatea de moderare a conținutului, oferind în același timp un punct de plecare solid pentru extinderi ulterioare.

Abstract

This thesis presents the design and development of the CourseHub web application, a platform that connects teachers and learners within a shared learning space. The main objective is to build a system in which a user can create a profile as either a teacher or a learner, search for a suitable partner by subject, level and role, initiate a collaboration through a collaboration request, and subsequently communicate in real time, including through file and teaching material sharing. The platform was developed using ASP.NET Core 8, Entity Framework Core and SQL Server, with real-time communication implemented through SignalR. Among the implemented features are secure authentication through an email OTP code, a role-based system built on Identity Roles with a dedicated admin panel, registration form protection through reCAPTCHA, avatar and file uploads validated by both type and size, and a user interface fully available in both Romanian and English. The results show a functional platform with an emphasis on security, user experience and content moderation capabilities, while also providing a solid foundation for future extensions.

Cuprins

1 Introducere **5**

1.1 Motivație 5
1.2 Context și preliminarii 5
1.3 Obiectivele lucrării 6
1.4 Contribuția personală 6
1.5 Structura lucrării 6

2 Analiza cerințelor **7**

2.1 Introducere 7
2.2 Actorii sistemului 7
2.3 Cerințe funcționale 7
2.4 Cerințe nefuncționale 8

3 Tehnologii și instrumente utilizate **9**

3.1 Introducere 9
3.2 ASP.NET Core 8 și Entity Framework Core 9
3.3 ASP.NET Identity și sistemul de roluri 9
3.4 SignalR – comunicare în timp real 10
3.5 reCAPTCHA 10
3.6 Localizare 10
3.7 Alte tehnologii și instrumente 10

4 Modelarea bazei de date și arhitectura sistemului **11**

4.1 Introducere 11
4.2 Modelarea bazei de date 11
4.2.1 Entități principale 11
4.2.2 Relații importante 12
4.3 Arhitectura logică a sistemului 12
4.3.1 Fluxuri funcționale principale 13

5 Implementarea funcționalităților majore **14**
5.1 Introducere 14
5.2 Gestionarea utilizatorilor 14
5.3 Autentificarea și verificarea prin OTP 15
5.4 Fluxul de completare a profilului 15
5.5 Căutarea și sistemul de cereri de colaborare 15
5.6 Mesageria în timp real și partajarea fișierelor 16
5.7 Panoul de administrare 16
5.8 Sistemul de localizare 17

6 Securitate **18**
6.1 Introducere 18
6.2 Autentificare și parole 18
6.3 Autorizare bazată pe roluri 19
6.4 Protecția formularului de înregistrare 19
6.5 Validarea fișierelor încărcate 20
6.6 Restricționarea accesului la conversații 20
6.7 Protecția împotriva atacurilor CSRF 20
6.8 Prevenirea falsificării expeditorului unui mesaj 21
6.9 Protecția integrității datelor și a rolului de administrator 21
6.10 Confidențialitatea datelor utilizatorilor 21

7 Testare **23**
7.1 Introducere 23
7.2 Generarea datelor de test 23
7.3 Scenarii testate manual 23
7.4 Depanarea unor probleme identificate prin testare 24

8 Concluzii și direcții de dezvoltare **25**

Bibliografie **26**

9 Declarație de autenticitate **28**

Capitolul 1

Introducere

1.1 Motivație

Când un elev lucrează unu la unu cu un profesor potrivit lui, învață mult mai bine decât la clasă. Problema e cum îl găsește. De obicei totul se rezolvă prin cunoștințe: întrebi pe cineva, care întreabă pe altcineva, și așa mai departe. Iar un profesor care vrea să dea meditații n-are unde să își facă un profil, să fie găsit de elevi și să vorbească apoi cu ei într-un singur loc.

Platformele generaliste de comunicare (rețele sociale, aplicații de mesagerie) nu oferă structura necesară unei relații profesor-elev: nu există un mod clar de a exprima ce predai sau ce vrei să înveți, nu există un sistem de conectare bazat pe cerere și acceptare, iar comunicarea și partajarea materialelor didactice se realizează ad-hoc, fără un istoric organizat pe fiecare colaborare. Există, așadar, o nevoie clară pentru o platformă dedicată, care să structureze întregul parcurs: de la crearea unui profil, la găsirea partenerului potrivit, la comunicarea propriu-zisă.

1.2 Context și preliminarii

Platformele educaționale existente se împart, în linii mari, în două categorii: platforme de cursuri înregistrate, fără interacțiune personalizată, și platforme de mediere între tutori și elevi, orientate comercial. Foarte puține pun accentul pe comunicarea directă, în timp real, odată ce colaborarea a fost stabilită, sau pe posibilitatea profesorului de a trimite un material de lucru chiar în conversația în care discută cu elevul.

CourseHub este gândit ca o soluție dedicată acestei nevoi: nu un magazin de cursuri, ci un spațiu în care oricine se poate înregistra ca persoană care vrea să predea sau să învețe, poate căuta un partener după criterii clare (rol, nivel, materie), poate iniția o colaborare printr-un mecanism explicit de cerere și acceptare, iar apoi poate comunica prin chat în timp real, inclusiv prin partajarea de fișiere.

1.3 Obiectivele lucrării

Obiectivul principal al acestei lucrări este proiectarea și implementarea platformei CourseHub, folosind ASP.NET Core 8 și tehnologii moderne asociate ecosistemului .NET. Obiectivele specifice sunt:

- construirea unui sistem de înregistrare securizat, cu verificare a adresei de email printr-un cod de unică folosință (OTP) și protecție a formularului de înregistrare printr-un mecanism de tip *CAPTCHA*;
- implementarea unui flux de completare a profilului (*wizard*) prin care utilizatorul își alege rolul (profesor sau elev), detaliile relevante rolului ales (nivel de experiență, respectiv nivel de studii) și materiile de interes;
- dezvoltarea unui sistem de căutare și filtrare a utilizatorilor după rol, nivel și materie;
- construirea unui mecanism explicit de conectare, bazat pe cereri de colaborare trimise de un elev unui profesor și acceptate sau respinse de acesta;
- implementarea unui sistem de mesagerie în timp real, folosind SignalR, cu suport pentru partajarea de fișiere de către profesori către elevii cu care colaborează;
- construirea unui sistem de roluri real, bazat pe *ASP.NET Identity Roles*, și a unui panou de administrare care permite gestionarea utilizatorilor, a materiilor disponibile pe platformă, precum și moderarea conținutului trimis prin chat;
- realizarea unei interfețe complet localizate, disponibilă atât în limba română, cât și în limba engleză.

1.4 Contribuția personală

Toate componentele platformei CourseHub – modelul de date, logica din controllere, interfața, autentificarea și verificarea prin OTP, sistemul de roluri, panoul de administrare, mesageria în timp real și localizarea – au fost proiectate și implementate integral ca parte a acestei lucrări, cu accent pe corectitudinea fluxurilor de autorizare, pe validarea riguroasă a datelor introduse de utilizator și pe o experiență de utilizare coerentă în ambele limbi.

1.5 Structura lucrării

Capitolul 2 analizează cerințele funcționale și nefuncționale, împreună cu actorii implicați, iar capitolul 3 prezintă tehnologiile folosite. Capitolul 4 descrie modelarea bazei de date și arhitectura sistemului, capitolul 5 detaliază implementarea, capitolul 6 aspectele de securitate, iar capitolul 7 metodologia de testare. Capitolul 8 conține concluziile, limitările și direcțiile de dezvoltare ulterioară.

Capitolul 2

Analiza cerințelor

2.1 Introducere

Înainte să încep să scriu cod, a trebuit să stabilesc cine folosește platforma și ce trebuie să poată face fiecare. În capitolul ăsta trec prin tipurile de utilizatori și prin cerințele care au ieșit din problema descrisă mai devreme.

2.2 Actorii sistemului

Platforma distinge patru tipuri de actori, ultimele trei reprezentate tehnic prin roluri reale din *ASP.NET Identity*: **Vizitator** (persoană neautentificată, poate accesa pagina principală, se poate înregistra sau autentifica), **Elev** (rol `Learn`, caută un profesor, trimite cereri de colaborare, comunică prin chat cu profesorii cu care colaborează), **Profesor** (rol `Teach`, poate fi găsit de elevi, acceptă sau respinge cereri de colaborare, poate trimite fișiere elevilor cu care colaborează) și **Administrator** (rol `Admin`, gestionează conturile, blocările și materiile disponibile, moderează mesajele).

Fiecare cont primește, în plus, rolul implicit `User` la înregistrare. Acesta nu influențează nicio funcționalitate vizibilă – există exclusiv pentru vizibilitate administrativă, ca un administrator să poată confirma din tabela de roluri că un cont este un membru înregistrat obișnuit.

2.3 Cerințe funcționale

- Înregistrare cu nume de utilizator, email și parolă; contul rămâne neconfirmat până la introducerea corectă a unui cod OTP trimis pe email.
- După confirmare, utilizatorul parcurge un flux de completare a profilului în trei pași: alegerea rolului, detaliile specifice rolului (vârstă, gen, nivel de studii/experiență) și selectarea a până la trei materii de interes.
- Căutarea altor utilizatori după nume, rol, nivel și materie.

- Trimiterea unei cereri de colaborare de la elev către profesor; acceptare, respingere sau anulare (cât timp e în așteptare) a cererii.
- Comunicare prin chat în timp real între utilizatorii care colaborează, cu posibilitatea profesorului de a atașa fișiere (documente, imagini, prezentări) conversației, și un indicator de mesaje necitite în bara de navigare.
- Editarea profilului propriu (biografie, avatar, materii, nivel); modificările sensibile (nume de utilizator, email) necesită reconfirmare prin cod OTP.
- Administrare: promovare/retrogradare a rolului de administrator, blocare temporară (1 zi, 7 zile, 1 lună) sau permanentă, ștergere de cont, gestionarea materiilor disponibile.

2.4 Cerințe nefuncționale

- **Securitate**: parolele nu sunt stocate în clar (gestionate de ASP.NET Identity ca *hash*); fișierele încărcate sunt validate după extensie și dimensiune; acțiunile administrative sunt restricționate exclusiv rolului `Admin`, verificat de framework, nu doar ascuns din interfață; toate cererile care modifică starea aplicației sunt însoțite de un token antiforgery validat pe server.
- **Localizare**: interfața este disponibilă integral în română și engleză, comutabilă fără reîncărcare manuală și fără autentificare.
- **Utilizabilitate**: interfața este responsivă și accesibilă atât pe desktop, cât și pe dispozitive mobile.
- **Extensibilitate**: arhitectura modulară (controllere separate pentru căutare, colaborări, cereri, chat, administrare) permite adăugarea de noi funcționalități fără a afecta modulele existente.
- **Fiabilitate**: acțiunile ireversibile (ștergere, blocare permanentă) sunt limitate prin verificări explicite – un administrator nu își poate șterge sau bloca propriul cont, iar niciun administrator nu poate fi blocat sau șters de altul.
- **Mentenabilitate**: evoluția schemei bazei de date este gestionată exclusiv prin migrații Entity Framework Core, trasabile și reproductibile pe orice mediu nou.

Capitolul 3

Tehnologii și instrumente utilizate

3.1 Introducere

Platforma CourseHub a fost construită folosind ecosistemul .NET, în jurul framework-ului *ASP.NET Core 8*. Această secțiune prezintă principalele tehnologii folosite și motivele pentru care au fost alese.

3.2 ASP.NET Core 8 și Entity Framework Core

ASP.NET Core este un framework *cross-platform* și *open-source*, dezvoltat de Microsoft, dedicat construirii de aplicații web și servicii API [7]. Aplicația CourseHub folosește modelul *MVC* (*Model-View-Controller*), un tipar arhitectural propus inițial pentru interfețe grafice și adaptat ulterior aplicațiilor web, pentru partea principală a aplicației și *Razor Pages* pentru fluxurile legate de autentificare (`Areas/Identity`), respectând convenția folosită de șabloanele oficiale ale platformei Identity. Structura proiectului urmează organizarea standard `Models/Controllers/Views`, completată de un director `Services` pentru componente reutilizabile (email, verificarea reCAPTCHA, curățarea codurilor OTP expirate). Limbajul folosit este C# [9].

Persistența datelor este realizată prin *Entity Framework Core*, ORM-ul oficial pentru .NET [10]. Toate entitățile platformei (`ApplicationUser`, `Message`, `HireRequest`, `Topic`, `OtpCode`) sunt mapate pe o bază de date *SQL Server* [14], iar evoluția schemei este gestionată exclusiv prin *migrații*: fiecare modificare a modelului de date este însoțită de o migrație dedicată, aplicată prin comanda `Update-Database`.

3.3 ASP.NET Identity și sistemul de roluri

Autentificarea și gestionarea utilizatorilor sunt realizate prin *ASP.NET Identity* [12], extins cu suport pentru roluri reale (`RoleManager<IdentityRole>`) [13]: `User` (atribuit automat oricărui cont), `Teach`, `Learn` și `Admin`. Rolurile `Teach`/`Learn` sunt menținute

sincronizat atât în câmpul `Role` al entității `ApplicationUser` (folosit de restul aplicației pentru afișare și filtrare), cât și în tabela reală `AspNetRoles`, ceea ce permite verificări stricte de tipul `[Authorize(Roles = "Admin")]` pentru panoul de administrare.

3.4 SignalR – comunicare în timp real

Chat-ul dintre profesor și elev este implementat cu *SignalR*, biblioteca oficială pentru comunicare bidirecțională în timp real peste *WebSockets* [8, 4], cu *fallback* automat la alte transporturi când este necesar. `ChatHub` expune metoda `SendMessage`, apelată de client prin conexiunea SignalR, care redirecționează mesajul instantaneu către destinatar, în paralel cu persistarea acestuia printr-un apel API separat, ale cărui endpoint-uri urmează principiile arhitecturale REST [5].

3.5 reCAPTCHA

Formularul de înregistrare este protejat împotriva înregistrărilor automate (*bot*) prin *Google reCAPTCHA v2* [6]. Verificarea token-ului trimis de client este realizată exclusiv pe server, printr-un apel către serviciul `siteverify` al Google, înainte ca un cont nou să fie creat efectiv.

3.6 Localizare

Interfața este disponibilă integral în română și engleză, folosind sistemul nativ de localizare din ASP.NET Core (`IStringLocalizer`, `IViewLocalizer` și fișiere de resurse `.resx`) [11]. Limba română este cea implicită; comutarea se face printr-un *cookie* setat de un controller dedicat, fără reîncărcare manuală și fără autentificare.

3.7 Alte tehnologii și instrumente

- **SMTP** pentru codurile OTP și notificările prin email;
- **Bootstrap 5** [1], peste care a fost construit un sistem propriu de stiluri (culori, carduri, animații);
- **Git** [16] și **Visual Studio** pentru controlul versiunilor, dezvoltare și depanare;
- **SQL Server LocalDB** ca sistem de gestiune a bazei de date în mediul de dezvoltare;
- **Postman**, pentru testarea manuală a endpoint-urilor API (de exemplu `/api/messages/upload-file`), independent de formular;
- **LaTeX** (XeLaTeX), pentru redactarea prezentei lucrări, necesar pentru suportul diacriticelor prin `fontspec` și `polyglossia`.

Capitolul 4

Modelarea bazei de date și arhitectura sistemului

4.1 Introducere

Aici arăt cum arată baza de date din spatele aplicației: ce entități am definit, cum se leagă între ele și cum am împărțit codul pe module.

4.2 Modelarea bazei de date

Baza de date este construită pe un model relațional [2], gestionat prin Entity Framework Core. Pe lângă entitățile proprii aplicației, schema include și tabelele standard generate de *ASP.NET Identity* (`AspNetUsers`, `AspNetRoles` etc.), entitatea `ApplicationUser` extinzând clasa `IdentityUser`.

4.2.1 Entități principale

Denumirile claselor păstrează terminologia engleză folosită la implementare (`Topic` pentru materie, `HireRequest` pentru cererea de colaborare), în timp ce textul lucrării și interfața folosesc termenii românești.

ApplicationUser reprezintă un cont de utilizator; extinde `IdentityUser` cu câmpurile `Bio`, `Role` (`"Teach"` sau `"Learn"`), `EducationLevel`, `TeachingLevel`, `Age`, `Gender`, `SelectedTopics` (identificatorii materiilor alese, separați prin virgulă), `AvatarPath` și `BannedUntil` (data până la care contul e blocat).

Message reprezintă un mesaj din conversația dintre doi utilizatori: conține `SenderId`, `ReceiverId`, `Content`, `Timestamp`, `IsRead`, precum și câmpurile opționale `FilePath`, `FileName` și `FileSize`, populate atunci când mesajul conține un fișier atașat.

HireRequest reprezintă o cerere de colaborare trimisă de un elev unui profesor: conține `RequesterId`, `TargetId`, `IsAccepted` și `RequestedAt`.

Topic reprezintă o materie (de exemplu, Matematică, Programare); conține `Name` și `Description`.

OtpCode reprezintă un cod de unică folosință generat pentru verificarea email-ului sau confirmarea unei modificări sensibile a profilului; conține `UserId`, `Code` și `ExpirationTime`.

4.2.2 Relații importante

Figura 4.1 prezintă relațiile principale dintre entități.

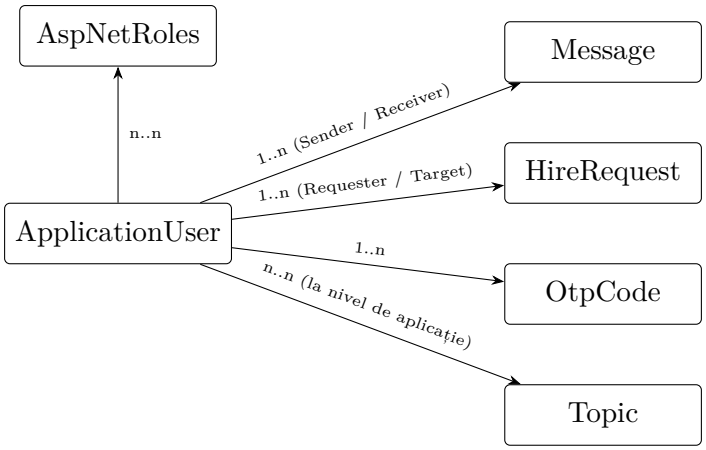


Figura 4.1: Diagrama entitate-relație simplificată a bazei de date CourseHub

Un utilizator (`ApplicationUser`) poate fi atât expeditor, cât și destinatar al mai multor mesaje, atât solicitant (elev), cât și țintă (profesor) a mai multor cereri de colaborare, și poate avea mai multe coduri OTP și roluri asociate. Dintre acestea, doar legătura cu `HireRequest` este materializată printr-o cheie externă propriu-zisă; relația cu `OtpCode` este menținută la nivel de aplicație, prin câmpul `UserId`.

Relația dintre `ApplicationUser` și `Topic` este, de asemenea, implementată intenționat la nivel de aplicație, prin lista de identificatori din `SelectedTopics`, și nu printr-o tabelă de legătură: un utilizator poate selecta cel mult trei materii, iar operațiile necesare sunt suficient de simple pentru a nu justifica o entitate suplimentară.

4.3 Arhitectura logică a sistemului

CourseHub este structurată modular, fiecare controller acoperind o arie de responsabilitate clară:

- **Identity (Areas/Identity)**: înregistrare, autentificare, verificare OTP, resetarea parolei, gestionarea profilului propriu (`Manage`);

- **SearchController**: căutarea și filtrarea utilizatorilor;
- **HireController**: trimiterea, acceptarea, anularea cererilor de colaborare;
- **ConnectionsController**: afișarea listei de colaborări active (elevi, respectiv profesori, în funcție de rol);
- **ChatController** și **ChatHub**: afișarea istoricului unei conversații și, respectiv, livrarea mesajelor în timp real prin SignalR;
- **MessagesController**: lista mesajelor primite, cu conversațiile recente ale utilizatorului;
- **AdminController**: panoul de administrare (utilizatori, roluri, blocări, materii, moderarea mesajelor).

Figura 4.2 ilustrează fluxul general al unei cereri prin sistem.

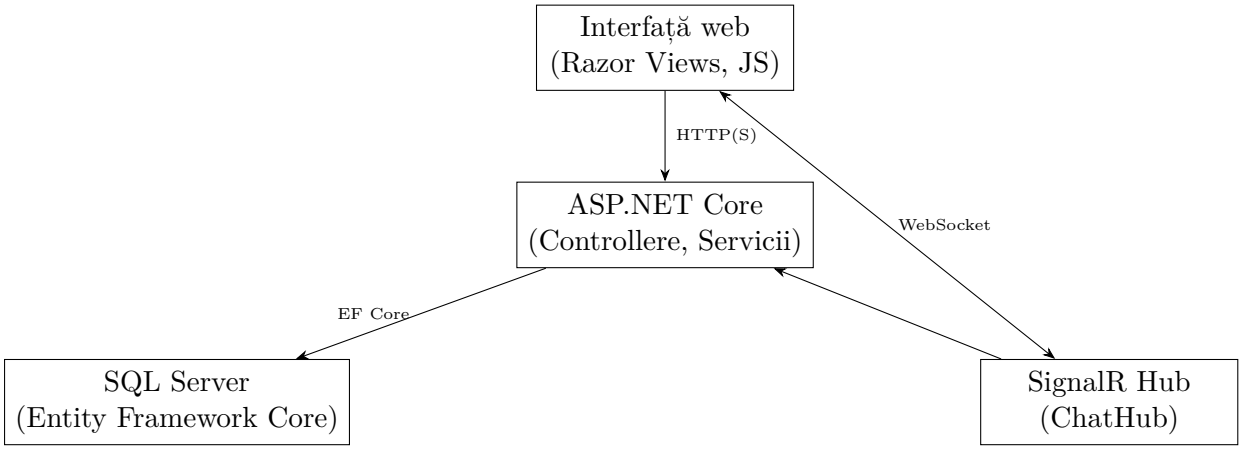


Figura 4.2: Arhitectura logică a platformei CourseHub

4.3.1 Fluxuri funcționale principale

Parcursul unui utilizator leagă modulele de mai sus într-un singur lanț: se înregistrează printr-un formular protejat de reCAPTCHA și își confirmă contul cu un cod OTP, parcurge apoi fluxul de trei pași prin care își definește rolul, detaliile și materiile de interes, caută un partener potrivit și îi trimite o cerere de colaborare, iar din momentul acceptării cei doi comunică prin chat în timp real, profesorul putând atașa suplimentar fișiere. În paralel, administratorul gestionează conturile, rolurile, blocările și materiile disponibile pe platformă.

Capitolul 5

Implementarea funcționalităților majore

5.1 Introducere

Acest capitol detaliază implementarea principalelor funcționalități ale platformei CourseHub: gestionarea conturilor și autentificarea, fluxul de completare a profilului, căutarea și sistemul de cereri de colaborare, mesageria în timp real cu partajare de fișiere, panoul de administrare și sistemul de localizare.

5.2 Gestionarea utilizatorilor

Entitatea `ApplicationUser` extinde clasa `IdentityUser` din ASP.NET Identity cu proprietăți specifice domeniului:

```
public class ApplicationUser : IdentityUser
{
    public string? Bio { get; set; }
    public string? Role { get; set; }
    public string? EducationLevel { get; set; }
    public string? TeachingLevel { get; set; }
    public int? Age { get; set; }
    public string? Gender { get; set; }
    public string? SelectedTopics { get; set; }
    public string? AvatarPath { get; set; }
    public DateTime? BannedUntil { get; set; }
}
```

Listing 5.1: Extinderea entității de utilizator (fragment)

Câmpul `BannedUntil` ilustrează o decizie de proiectare deliberată: în loc de un câmp boolean separat (`IsBanned`), o singură dată nullable acoperă toate cazurile – `null` înseamnă cont neblocat, o dată din viitor înseamnă blocare temporară, iar

`DateTime.MaxValue` este folosit convențional pentru blocare permanentă. Verificarea „este blocat” se reduce, în orice context, la o singură comparație: `BannedUntil > DateTime.UtcNow`.

5.3 Autentificarea și verificarea prin OTP

Înregistrarea unui cont nou nu autentifică imediat utilizatorul. După crearea contului, se generează un cod numeric de 6 cifre, stocat împreună cu un termen de expirare în tabela `OtpCode`, iar codul este trimis pe adresa de email indicată. Contul rămâne neconfirmat (`EmailConfirmed = false`) până la introducerea corectă a codului pe pagina dedicată de verificare. Același mecanism este reutilizat pentru confirmarea modificărilor sensibile ale profilului (schimbarea numelui de utilizator sau a adresei de email).

5.4 Fluxul de completare a profilului

Imediat după confirmarea contului, utilizatorul este direcționat către un flux de completare a profilului, organizat în trei pași: alegerea rolului, completarea detaliilor specifice rolului ales și selectarea materiilor de interes. O atenție particulară a fost acordată transmiterii corecte a rolului între pași – valoarea aleasă la primul pas trebuie păstrată într-un câmp ascuns al formularului, altfel serverul nu ar putea decide dacă trebuie salvat nivelul de educație sau nivelul de predare. La finalul primului pas, rolul ales este scris atât în câmpul `Role` al utilizatorului, cât și în tabela reală de roluri Identity:

```
user.Role = Wizard.Role;
await _userManager.UpdateAsync(user);

var otherRole = Wizard.Role == "Teach" ? "Learn" : "Teach";
if (await _userManager.IsInRoleAsync(user, otherRole))
    await _userManager.RemoveFromRoleAsync(user, otherRole);
if (!await _userManager.IsInRoleAsync(user, Wizard.Role))
    await _userManager.AddToRoleAsync(user, Wizard.Role);
```

Listing 5.2: Sincronizarea rolului domeniu cu rolul Identity

5.5 Căutarea și sistemul de cereri de colaborare

Căutarea permite filtrarea utilizatorilor după nume, rol, nivel de experiență și materie, excluzând întotdeauna conturile cu rol de administrator din rezultate, indiferent de valoarea câmpului `Role` pe care acestea o au. Un elev poate trimite unei persoane cu rol de profesor o cerere de colaborare (`HireRequest`); profesorul o poate accepta sau respinge din pagina proprie de cereri, iar elevul poate vizualiza starea propriilor cereri

trimise și le poate anula cât timp sunt în așteptare. Odată o cerere acceptată, cei doi utilizatori devin parteneri și pot iniția o conversație.

5.6 Mesageria în timp real și partajarea fișierelor

Comunicarea dintre utilizatori se realizează printr-un chat cu livrare instantanee, implementat cu SignalR. Hub-ul expune o singură metodă, apelată de client pentru a trimite un mesaj text:

```
public class ChatHub : Hub
{
    public async Task SendMessage(
        string senderId, string receiverId, string message)
    {
        await Clients.User(receiverId)
            .SendAsync("ReceiveMessage", senderId, message);
    }
}
```

Listing 5.3: ChatHub – livrarea mesajelor în timp real

În paralel cu livrarea prin SignalR, clientul trimite mesajul și către un endpoint API dedicat, responsabil de persistarea acestuia în baza de date – separarea celor două căi permite ca istoricul conversației să rămână disponibil chiar dacă unul dintre utilizatori nu este conectat în momentul trimiterii.

Profesorii pot, suplimentar, atașa fișiere conversației. Restricția este aplicată exclusiv pe server, nu doar prin ascunderea butonului de atașare din interfață pentru elevi: tipul fișierului este verificat printr-o listă albă de extensii, dimensiunea este limitată, iar numele fișierului salvat pe disc este generat aleator (`Guid.NewGuid`), pentru a preveni suprascrierea accidentală a unui fișier existent cu același nume.

5.7 Panoul de administrare

Panoul de administrare este restricționat prin atributul `[Authorize(Roles = "Admin")]`, aplicat la nivel de controller – framework-ul respinge orice cerere provenită de la un utilizator neautorizat, înainte ca vreo linie de cod din interior să se execute.

Administratorul poate promova sau retrograda alți utilizatori, poate șterge un cont (împreună cu toate mesajele, cererile de colaborare, codurile OTP și avatarul asociate) și poate bloca temporar sau permanent un cont:

```
switch (duration)
{
    case "1d": user.BannedUntil = DateTime.UtcNow.AddDays(1); break;
    case "7d": user.BannedUntil = DateTime.UtcNow.AddDays(7); break;
```

```
    case "1m": user.BannedUntil = DateTime.UtcNow.AddMonths(1); break;
    case "permanent": user.BannedUntil = DateTime.MaxValue; break;
    case "unban": user.BannedUntil = null; break;
}
```

Listing 5.4: Aplicarea unei blocări (fragment din AdminController)

Un administrator nu se poate bloca sau șterge pe sine, iar niciun administrator nu poate fi blocat sau șters de un alt administrator, pentru a preveni pierderea accidentală a accesului la panoul de administrare. Pentru moderare, administratorul poate vizualiza, pentru orice utilizator, lista completă a mesajelor trimise de acesta.

5.8 Sistemul de localizare

Textele din interfață sunt extrase în fișiere de resurse (`.resx`), organizate în oglindă cu structura de directoare a vizualizărilor, iar textul românesc este folosit direct drept cheie de resursă:

```
<h1>@Localizer["Panou de administrare"]</h1>
```

Listing 5.5: Exemplu de utilizare a localizării într-o vizualizare

Limba activă este determinată printr-un *cookie* setat de un controller dedicat, apelat la comutarea de limbă din bara de navigare; schimbarea limbii nu necesită autentificare și este păstrată între sesiuni.

Capitolul 6

Securitate

6.1 Introducere

Securitatea nu e o funcționalitate pe care o adaugi la final, ci ceva ce trebuie avut în minte peste tot. De aceea, pentru fiecare formular, încărcare de fișier și acțiune de administrare m-am întrebat același lucru: ce s-ar întâmpla dacă cineva ar apela-o direct, fără să treacă prin interfață [15]?

6.2 Autentificare și parole

Parolele nu sunt niciodată stocate în clar; gestionarea lor este delegată componentei `PasswordHasher` din ASP.NET Identity [12], care aplică *hashing* cu *salt* unic per utilizator, în linie cu recomandările privind gestionarea autentificatorilor de tip memorat-secret publicate de NIST [17].

Autentificarea unui cont blocat este respinsă înainte ca sesiunea să fie creată. Pentru a evita confirmarea existenței unui cont blocat unui atacator care nu cunoaște parola corectă, verificarea parolei se face separat de autentificarea efectivă – mesajul de blocare apare exclusiv când parola introdusă este corectă:

```
var passwordCheck = await _signInManager.CheckPasswordSignInAsync(
    user, Input.Password, lockoutOnFailure: false);

if (passwordCheck.Succeeded)
{
    if (user.BannedUntil.HasValue &&
        user.BannedUntil.Value > DateTime.UtcNow)
    {
        ModelState.AddModelError(string.Empty, banMessage);
        return Page();
    }
    await _signInManager.SignInAsync(user, isPersistent: false);
    return LocalRedirect(returnUrl);
```

```
}
```

Listing 6.1: Verificarea blocării unui cont, ulterior validării parolei

6.3 Autorizare bazată pe roluri

Panoul de administrare, precum și fiecare acțiune sensibilă din `AdminController` (blocare, ștergere, promovare la rolul de administrator, gestionarea materiilor) sunt protejate prin `[Authorize(Roles = "Admin")]` [13], verificat de *middleware*-ul de autorizare al ASP.NET Core, înaintea execuției oricărui cod din controller. Aceeași filosofie a fost aplicată restricțiilor de domeniu care nu se mapează pe un rol Identity: doar un utilizator cu `Role = "Teach"` poate atașa fișiere unei conversații, verificare realizată în endpoint-ul de încărcare.

6.4 Protecția formularului de înregistrare

Formularul de înregistrare este protejat prin *Google reCAPTCHA v2* [6]. Token-ul generat de widget-ul din pagină este trimis către server și verificat printr-un apel către serviciul oficial de validare al Google, înainte ca vreun cont să fie creat:

```
public async Task<bool> VerifyAsync(
    string? recaptchaResponse, string? remoteIp)
{
    if (string.IsNullOrWhiteSpace(recaptchaResponse))
        return false;

    var response = await client.PostAsync(
        "https://www.google.com/recaptcha/api/siteverify",
        new FormUrlEncodedContent(parameters));

    var json = await response.Content.ReadAsStringAsync();
    using var doc = JsonDocument.Parse(json);
    return doc.RootElement.TryGetProperty("success", out var s)
           && s.GetBoolean();
}
```

Listing 6.2: Verificarea token-ului reCAPTCHA (fragment)

Dacă cheia secretă nu este configurată, verificarea eșuează implicit (*fail closed*) – înregistrarea rămâne blocată până la configurarea corectă, în loc să permită tuturor înregistrărilor să treacă nevalidate.

6.5 Validarea fișierelor încărcate

Atât avatarele de profil, cât și fișierele atașate conversațiilor sunt validate pe server, nu doar prin atributul `accept` al elementului HTML de tip fișier (ușor de ocolit): tipul este verificat printr-o listă albă de extensii, dimensiunea este limitată explicit, iar numele fișierului salvat pe disc este generat aleator, pentru a preveni atât suprascrierea accidentală a unui fișier existent, cât și deducerea, din nume, a proveniență acestuia.

6.6 Restricționarea accesului la conversații

O conversație există numai între doi utilizatori legați printr-o cerere de colaborare acceptată. Interfața respectă regula în mod natural – butonul de chat apare exclusiv în lista de colaborări –, dar ea nu poate rămâne doar acolo: un utilizator autentificat ar putea deschide manual adresa unei conversații cu orice cont, indicând identificatorul acestuia în URL. Verificarea este, prin urmare, repetată pe server:

```
var areConnected = await _context.HireRequests.AnyAsync(h =>
    h.IsAccepted &&
    ((h.RequesterId == currentUserId && h.TargetId == userId) ||
     (h.RequesterId == userId && h.TargetId == currentUserId)));

if (!areConnected) return Forbid();
```

Listing 6.3: Verificarea existenței unei colaborări acceptate

Aceeași condiție este aplicată atât în endpoint-ul care persistă un mesaj text, cât și în cel de încărcare a fișierelor.

6.7 Protecția împotriva atacurilor CSRF

Un atac de tip *Cross-Site Request Forgery* exploatează faptul că browser-ul atașează automat *cookie*-ul de sesiune oricărei cereri către aplicație, inclusiv uneia inițiate de pe un site străin: o pagină ostilă poate trimite, în numele unui administrator autentificat, o cerere de ștergere a unui cont [15].

Contramăsura standard este un *token* unic per sesiune, inclus în fiecare formular și validat la primirea cererii. În ASP.NET Core, paginile Razor îl validează automat, în timp ce controllerele MVC nu o fac decât dacă li se cere explicit – exact categoria de acțiuni sensibile din `AdminController` și `HireController`. Validarea a fost, de aceea, activată global:

```
builder.Services.AddAntiforgery(options =>
    options.HeaderName = "RequestVerificationToken");

builder.Services.AddControllersWithViews(options =>
```

```
{
    options.Filters.Add(new AutoValidateAntiforgeryTokenAttribute());
});
```

Listing 6.4: Activarea globală a validării token-ului antiforgery

Formularele aplicației sunt generate prin *tag helper*-ul de formular, care include deja token-ul ca un câmp ascuns, deci nicio vizualizare nu a trebuit modificată. Singurele două cereri care nu provin dintr-un formular – trimiterea unui mesaj și încărcarea unui fișier – transmit token-ul printr-un antet HTTP.

6.8 Prevenirea falsificării expeditorului unui mesaj

Endpoint-ul API prin care se persistă un mesaj nu are voie să aibă încredere în identificatorul de expeditor trimis de client; acesta este întotdeauna suprascris cu identitatea utilizatorului autentificat:

```
var senderId = _userManager.GetUserId(User);
if (senderId == null) return Unauthorized();

message.SenderId = senderId;
```

Listing 6.5: Suprascrierea expeditorului cu identitatea autentificată

Fără această verificare, un utilizator autentificat ar fi putut construi manual o cerere HTTP care să pară provenită de la un alt cont.

6.9 Protecția integrității datelor și a rolului de administrator

Ștergerea unui cont elimină explicit toate înregistrările care fac referire la acel utilizator – mesajele trimise și primite, cererile de colaborare și codurile OTP – înainte de ștergerea efectivă a contului, evitând înregistrările orfane.

Un administrator nu se poate retrograda, bloca sau șterge pe sine, iar niciun administrator nu poate fi blocat sau șters de un altul, pentru ca platforma să nu rămână, accidental sau prin intervenție rău-intenționată, fără niciun cont capabil să acceseze panoul.

6.10 Confidențialitatea datelor utilizatorilor

Platforma stochează date cu caracter personal (nume, email, biografie, avatar, conținutul mesajelor), motiv pentru care principiile de minimizare prevăzute de Regulamentul General privind Protecția Datelor [3] au fost avute în vedere la proiectare:

utilizatorul își poate elimina avatarul oricând, iar ștergerea unui cont înlătură și fișierul de pe disc, nu doar rândul din baza de date. Codurile OTP expiră în 10 minute și sunt șterse automat, reducând fereastra în care ar putea fi exploatate dacă baza de date ar fi compromisă.

Capitolul 7

Testare

7.1 Introducere

Am testat aplicația manual, pe măsură ce am scris-o, mergând pe scenarii. Problema e că multe fluxuri au nevoie de mai mulți utilizatori: nu poți testa căutarea, cererile de colaborare sau o conversație cu un singur cont. Ca să nu înregistrez de fiecare dată zeci de conturi și să trec prin OTP pentru fiecare, am scris un serviciu care îmi generează datele de test la pornire.

7.2 Generarea datelor de test

Serviciul rulează doar când mediul de execuție este `Development`, ca să nu ajungă niciodată pe un server real. La pornire creează conturi de profesori și elevi cu profilul completat: vârstă, gen, nivel, materii. Nu scriu conturile direct în baza de date, ci le trec prin `UserManager`, ca parolele să fie procesate de `PasswordHasher` și rolurile Identity să fie atribuite corect – altfel conturile de test s-ar comporta diferit de cele reale, iar testul n-ar mai însemna nimic. Adaug și câteva cereri de colaborare acceptate, cu conversații între ele.

Înainte să creeze un cont, serviciul verifică după email dacă există deja. Așa pot porni aplicația de câte ori vreau fără să se dubleze nimic.

7.3 Scenarii testate manual

- **Înregistrare și verificare**: date valide, cod OTP corect și incorect, expirarea codului, formular trimis fără reCAPTCHA;
- **Completarea profilului**: cei trei pași pentru ambele roluri, revenirea la un pas anterior cu păstrarea datelor introduse;
- **Căutare și filtrare**: filtrare după nume, rol, nivel și materie, izolat și combinat, precum și excluderea conturilor de administrator din rezultate;

- **Cereri de colaborare**: trimitere, acceptare, respingere, anulare, precum și încercarea unei a doua cereri către același profesor;
- **Mesagerie**: livrare instantanee a unui mesaj text, trimiterea unui fișier de către un cont cu rol de profesor și verificarea faptului că un elev nu are acces la această funcționalitate, inclusiv prin apelarea directă a endpoint-ului API, ocolind interfața;
- **Panou de administrare**: promovarea și retrogradarea unui administrator, imposibilitatea unui administrator de a se bloca sau șterge pe sine, blocarea temporară și permanentă, precum și verificarea faptului că un cont blocat nu se mai poate autentifica;
- **Localizare**: comutarea între română și engleză pe fiecare pagină, cu păstrarea limbii alese între navigări succesive.

7.4 Depanarea unor probleme identificate prin testare

Testând iar și iar aceleași fluxuri am dat peste trei probleme pe care nu le văzusem scriind codul:

- la primul pas al fluxului de completare a profilului, rolul ales (`Wizard.Role`) nu era transmis prin formular la pasul al doilea, fiind folosit doar ca să decid ce câmpuri arăt pe ecran. Așa că la salvare aplicația scria mereu în nivelul de predare, oricare ar fi fost rolul ales. Am rezolvat punând un câmp ascuns care duce rolul mai departe, de la un pas la altul;
- controllerul responsabil de afișarea listei de colaborări nu era protejat cu `[Authorize]`, cum erau toate celelalte. Dacă intrai pe pagină fără să fii autentificat, primeai o eroare în loc să fii trimis la pagina de autentificare. Am adăugat atributul care lipsea;
- am încercat să intru pe pagini scriind adresa direct în browser, nu apăsând butoanele. Așa am văzut că puteam deschide o conversație cu orice cont, doar punându-i identificatorul în URL, chiar dacă nu aveam nicio colaborare cu el. Din interfață nu ajungi acolo decât din lista de colaborări, dar asta nu e o protecție. Am mutat verificarea pe server, și pentru afișarea istoricului, și pentru cele două endpoint-uri prin care se scrie un mesaj.

Toate trei au ieșit la iveală abia când am încercat aplicația din poziția altui rol decât cel la care mă gândisem când scriam codul, sau când am ocolit interfața.

Capitolul 8

Concluzii și direcții de dezvoltare

În lucrarea asta am proiectat și am construit CourseHub, o aplicație web pe ASP.NET Core 8 prin care profesorii și elevii se găsesc și lucrează împreună. Aplicația acoperă tot drumul unui utilizator: se înregistrează și își confirmă contul prin OTP, își completează profilul, caută pe cineva potrivit, trimite o cerere de colaborare și, după ce e acceptată, discută prin chat în timp real și își trimit fișiere. În spate există un sistem real de roluri Identity, peste care am construit panoul de administrare, iar toată interfața merge în română și în engleză.

Sunt și lucruri pe care nu am apucat să le fac. Aplicația rulează pe o bază de date locală, nu am publicat-o în cloud. Notificările de mesaje noi se iau prin interogare periodică (*polling*), nu instantaneu, iar un fișier trimis în conversație se vede la celălalt abia după ce dă refresh. Nu am scris teste automate. Iar dacă blochez un cont care e deja autentificat, sesiunea lui rămâne activă până iese singur.

Dacă aș continua proiectul, aș lua-o în ordinea asta:

- introducerea unei suite de teste automate, pentru a reduce riscul de regresie pe măsură ce platforma crește;
- invalidarea imediată a sesiunilor active la blocarea unui cont;
- extinderea livrării în timp real prin SignalR și pentru mesajele cu fișiere atașate, eliminând reîncărcarea paginii;
- publicarea platformei într-un mediu cloud, cu o bază de date gestionată și integrare continuă;
- organizarea materialelor partajate prin conversații într-o bibliotecă per colaborare;
- introducerea unui sistem de recomandare a partenerilor potriviți, bazat pe materiile de interes și pe istoricul colaborărilor.

În concluzie, CourseHub demonstrează fezabilitatea unei platforme dedicate conectării profesorilor cu elevii, cu un accent particular pe securitate – autorizare aplicată consecvent la nivel de server, validare riguroasă a conținutului încărcat și moderare funcțională.

Bibliografie

[1] Bootstrap Team, *Bootstrap v5.1 Documentation*, Ultima accesare: 1 septembrie 2026., 2024, URL: https://getbootstrap.com/docs/5.1/getting-started/introduction/ (accesat în 1.9.2026).

[2] Ramez Elmasri și Shamkant B. Navathe, *Fundamentals of Database Systems*, a 7-a ed., Pearson, 2015, ISBN: 978-0-13-397077-7.

[3] European Parliament and Council of the European Union, *Regulation (EU) 2016/679 (General Data Protection Regulation)*, Ultima accesare: 1 septembrie 2026., 2016, URL: https://eur-lex.europa.eu/eli/reg/2016/679/oj (accesat în 1.9.2026).

[4] Ian Fette și Alexey Melnikov, *The WebSocket Protocol*, rap. teh. RFC 6455, Internet Engineering Task Force, 2011, DOI: 10.17487/RFC6455.

[5] Roy Thomas Fielding, „Architectural Styles and the Design of Network-based Software Architectures", Ultima accesare: 1 septembrie 2026., Teză de doct., University of California, Irvine, 2000, URL: https://ics.uci.edu/~fielding/pubs/dissertation/fielding_dissertation.pdf (accesat în 1.9.2026).

[6] Google, *reCAPTCHA v2 Documentation*, Ultima accesare: 1 septembrie 2026., 2024, URL: https://developers.google.com/recaptcha/docs/display (accesat în 1.9.2026).

[7] Microsoft, *ASP.NET Core Documentation*, Ultima accesare: 1 septembrie 2026., 2024, URL: https://learn.microsoft.com/en-us/aspnet/core/ (accesat în 1.9.2026).

[8] Microsoft, *ASP.NET Core SignalR Documentation*, Ultima accesare: 1 septembrie 2026., 2024, URL: https://learn.microsoft.com/en-us/aspnet/core/signalr/introduction (accesat în 1.9.2026).

[9] Microsoft, *C# Documentation*, Ultima accesare: 1 septembrie 2026., 2024, URL: https://learn.microsoft.com/en-us/dotnet/csharp/ (accesat în 1.9.2026).

[10] Microsoft, *Entity Framework Core Documentation*, Ultima accesare: 1 septembrie 2026., 2024, URL: https://learn.microsoft.com/en-us/ef/core/ (accesat în 1.9.2026).

[11] Microsoft, *Globalization and localization in ASP.NET Core*, Ultima accesare: 1 septembrie 2026., 2024, URL: https://learn.microsoft.com/en-us/aspnet/core/fundamentals/localization (accesat în 1.9.2026).

[12] Microsoft, *Introduction to Identity on ASP.NET Core*, Ultima accesare: 1 septembrie 2026., 2024, URL: https://learn.microsoft.com/en-us/aspnet/core/security/authentication/identity (accesat în 1.9.2026).

[13] Microsoft, *Role-based authorization in ASP.NET Core*, Ultima accesare: 1 septembrie 2026., 2024, URL: https://learn.microsoft.com/en-us/aspnet/core/security/authorization/roles (accesat în 1.9.2026).

[14] Microsoft, *SQL Server Documentation*, Ultima accesare: 1 septembrie 2026., 2024, URL: https://learn.microsoft.com/en-us/sql/sql-server/ (accesat în 1.9.2026).

[15] OWASP Foundation, *OWASP Top 10 – The Ten Most Critical Web Application Security Risks*, Ultima accesare: 1 septembrie 2026., 2025, URL: https://owasp.org/www-project-top-ten/ (accesat în 1.9.2026).

[16] Software Freedom Conservancy, *Git Documentation*, Ultima accesare: 1 septembrie 2026., 2024, URL: https://git-scm.com/docs (accesat în 1.9.2026).

[17] David Temoshok, James L. Fenton, Yee-Yin Choong, Naomi B. Lefkovitz, Andrew R. Regenscheid, Ryan Galluzzo și Justin P. Richer, *Digital Identity Guidelines: Authentication and Authenticator Management*, rap. teh. NIST Special Publication 800-63B-4, National Institute of Standards and Technology, 2025, DOI: 10.6028/NIST.SP.800-63b-4.

(Acest document se leagă împreună cu lucrarea de licenţă /diploma/disertație)

DECLARAȚIE

Subsemnatul/Subsemnata AVRAMESCU ISAAC SEBASTIAN, candidat la examenul de **licență/diplomă/disertație**, sesiunea septembrie 2026, la **Facultatea de Matematică şi Informatică,** programul de studii INFORMATICA declar pe propria răspundere că lucrarea de faţă este rezultatul muncii mele, pe baza cercetărilor mele şi pe baza informaţiilor obţinute din surse care au fost citate şi indicate, conform normelor etice, în note şi în bibliografie. Declar că nu am folosit în mod tacit sau ilegal munca altora şi că nici o parte din teză nu încalcă drepturile de proprietate intelectuală ale altcuiva, persoană fizică sau juridică. Declar că lucrarea nu a mai fost prezentată sub această formă vreunei instituţii de învăţământ superior în vederea obţinerii unui grad sau titlu ştiinţific ori didactic.

Data **Semnătura,**

01.09.2026

